



مقدمه

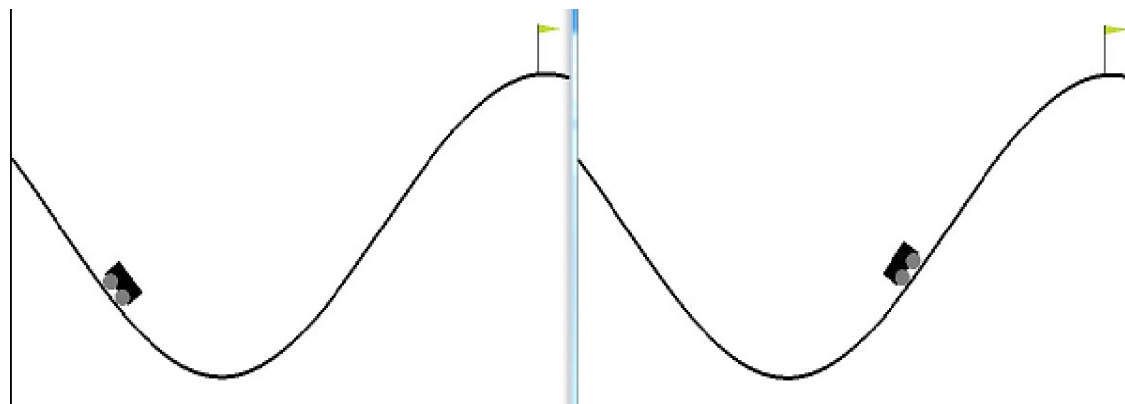
یادگیری تقویتی (Reinforcement Learning) یکی از شاخه‌های کلیدی هوش مصنوعی است که در آن یک عامل (Agent) یاد می‌گیرد از طریق تعامل با محیط و دریافت پاداش یا جریمه، تصمیمات بهینه‌ای اتخاذ کند. در این تمرین قرار است با استفاده از الگوریتم کلاسیک Q-Learning (روش Tabular) که در درس آموخته‌اید، یک عامل هوشمند برای حل محیط MountainCar پیاده‌سازی کنید.

در روش Tabular، یک جدول (Q-Table) نگهداری می‌شود که برای هر جفتِ حالت (State) و عمل (Action) یک مقدار عددی ذخیره می‌کند. این مقدار نشان‌دهنده کیفیت انجام آن عمل در آن حالت (پاداش تجمعی مورد انتظار در آینده) است. عامل با تعامل با محیط و دریافت پاداش، مقادیر جدول را طبق رابطه‌ی بیان شده در صفحه سوم به‌روزرسانی می‌کند.

هدف از این تمرین، آشنایی عملی با چالش‌های اساسی یادگیری تقویتی از جمله گسسته‌سازی فضای پیوسته (State Discretization)، ایجاد تعادل بین اکتشاف و بهره‌برداری (Exploration vs. Exploitation) و مهندسی پاداش (Reward Shaping) است.

توضیح مسئله

محیط "MountainCar-v0" (از کتابخانه OpenAI Gym) شامل یک خودروی ضعیف است که در دره‌ای بین دو تپه گیر افتاده است. موتور این خودرو آنقدر قدرت ندارد که مستقیماً از شیب سمت راست بالا برود و به پرچم (نقطه هدف) برسد. بنابراین، خودرو باید یاد بگیرد که گاهی باید به سمت تپه چپ نیز به‌صورت دنده عقب برود تا انرژی پتانسیل کافی برای صعود از تپه راست را به دست آورد.



طبق الگوریتم کلاسیک Q-learning، در هر مرحله، عامل (خودرو) حالت فعلی را مشاهده می‌کند، با استفاده از یک نوع سیاست (Policy) یک عمل (Action) را انتخاب می‌کند، آن عمل را در محیط اجرا می‌کند، پاداش (Reward) و حالت (State) بعدی را دریافت می‌کند، و سپس مقدار Q را به‌روز می‌کند.

فضای حالت (State Space)

مسئله دارای یک فضای پیوسته شامل دو متغیر زیر است:

- مکان خودرو (Position): بازه $[-1.2, 0.6]$
- سرعت خودرو (Velocity): بازه $[-0.07, 0.07]$

فضای عمل (Action Space)

در این محیط، سه عمل گسسته‌ی زیر برای عامل مجاز است:

- ۰: حرکت به چپ
- ۱: ثابت ماندن (موتور خاموش)
- ۲: حرکت به راست

تابع پاداش (Reward Function)

عامل به ازای هر گام زمانی (Step)، به طور ثابت پاداش ۱- دریافت می‌کند تا زمانی که به پرچم (مکان ۰.۵) برسد. این، یک مسئله با پاداش پراکنده (Sparse Reward) است که یادگیری را دشوار می‌کند.

پیاده‌سازی مسئله

در این تمرین، شما باید کلاس "QLearningAgent" را کامل کنید. بخش‌هایی که داخل کد مشخص شده‌اند، نیاز به تکمیل توسط شما دارند. همچنین هنگام پیاده‌سازی الگوریتم، به بخش سوالات نیز دقت کنید زیرا در آن‌ها ایده‌ها و راهنمایی‌هایی برای پیاده‌سازی بهتر الگوریتم وجود دارند.

طراحی عامل (Agent Implementation)

برای طراحی عامل و آماده‌سازی محیط، ابتدا لازم است که برخی از تعاریف زیر را پیاده‌سازی کنید.

گسسته‌سازی (Discretization)

در ابتدا باید فضای حالت پیوسته‌ی مربوط به مختصات خودرو (مکان و سرعت) را به یک فضای حالت گسسته تبدیل کنید.

انتخاب عمل (Action Selection)

عامل باید برای الگوریتم انتخاب عمل خود، از سیاست مبتنی بر Epsilon Greedy استفاده کند. لذا نیاز است تا این الگوریتم را پیاده‌سازی کنید.

به‌روزرسانی جدول (Q-table Updating) Q

از آنجا که عامل محیط را کامل نمی‌شناسد و گذارها (Transitions) و تابع دقیق آن را نمی‌داند، باید از رابطه‌ی به‌روزرسانی نمونه‌ای (Sample Update) و Model-Free بر اساس معادله بلمن (Bellman Equation) برای به‌روزرسانی Q و جدول آن استفاده کنید و آن را طبق این رابطه پیاده‌سازی کنید:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma (\max_{a'} Q(s', a')) - Q(s, a)]$$

به طوری که در این رابطه، α نرخ یادگیری (Learning Rate) و γ ضریب تنزیل (Discount Factor) می‌باشد.

آموزش عامل (Agent Training)

حال می‌توانید عامل را آموزش دهید و همچنین از خروجی آن برای تحلیل‌ها و رسم نمودار سه‌بعدی در بخش‌های بعدی استفاده کنید.

طبق استاندارد این محیط، حل کردن آن مستلزم کسب میانگین پاداش کل بیش از ۱۱۰- در ۱۰۰ اپیزود متوالی است. توجه کنید که با توجه به تعداد بالای اپیزودها، آموزش ممکن است چندین دقیقه (بسته به قدرت پردازنده شما) زمان ببرد. لذا لطفاً آن را اجرا کرده و تا پایان پردازش آن صبور باشید.

در طول آموزش، برای هر اپیزود مقدار پاداش محیط را ذخیره کنید. سپس دو نمودار زیر را روی یک شکل رسم کنید:

- نمودار امتیاز: پاداش واقعی کسب شده در هر اپیزود آموزشی را نشان می‌دهد.
- نمودار میانگین: میانگین پاداش واقعی در ۱۰۰ اپیزود اخیر را نشان می‌دهد.

همچنین نمودار مقدار ϵ در طول آموزش را نیز رسم کنید.

این نمودارها را در گزارش خود بیاورید و آن‌ها را، مخصوصاً از نظر همگرایی، پایداری و تراکم امتیازها، تحلیل کنید.

ارزیابی عامل (Agent Evaluation)

پس از اتمام فرآیند آموزش، باید عامل را بدون اکتشاف (Exploration) ارزیابی کنید. یعنی ϵ را برابر با صفر قرار دهید تا عامل همیشه عمل با بیشترین مقدار Q را انتخاب کند.

ارزیابی را برای ۱۰۰ اپیزود انجام دهید و میانگین، بهترین و بدترین امتیاز عامل را گزارش کنید.

تحلیل نرخ اکتشاف (Exploration vs. Exploitation)

یکی از چالش‌های اصلی در RL، تعادل بین اکتشاف (Exploration) و بهره‌برداری (Exploitation) است.

حداقل چهار استراتژی برای کاهش مقدار ϵ (ثابت، خطی، لگاریتمی و نمایی) را برای حداقل ۱۰۰۰۰۰ اپیزود آزمایش کنید و برای هر کدام، سه نمودار مربوط به بخش آموزش عامل را رسم کنید. و آن‌ها را در گزارش خود تحلیل کنید.

نفرین ابعاد و دقت گسسته‌سازی (Curse of Dimensionality)

در روش‌های Tabular، اندازه جدول Q بسیار حیاتی است. در این بخش باید اثر اندازه‌ی گسسته‌سازی را بررسی کنید و مدل را با چند اندازه باکت (Bucket) مختلف اجرا کنید و نتایج را مقایسه کنید.

بدین منظور، تعداد باکت‌ها را یک‌بار روی حالت خیلی کوچک (5×5) و یک‌بار نیز روی حالت خیلی بزرگ (100×100) تنظیم کنید و هر کدام را برای حداقل ۱۰۰۰۰۰ اپیزود اجرا کنید.

نتایج و مقایسه‌های این بخش را در گزارش خود بیاورید و آن‌ها را تحلیل کنید.

مصورسازی تابع ارزش (Value Function Visualization)

پس از آموزش کامل مدل، تابع ارزش را به صورت سه بعدی (3D Surface Plot) رسم کنید. دقت کنید که محور Z باید نشان‌دهنده‌ی $\max_a Q(s, a)$ باشد.

دقت کنید که در رسم این نمودار، فقط حالاتی را نمایش دهید که در طول آموزش حداقل چند بار توسط عامل مشاهده شده‌اند؛ چرا که حالات مشاهده نشده، مقدار Q قابل اعتمادی ندارند و نباید در تحلیل نتایج ارزش لحاظ شوند.

این نمودار را در گزارش خود بیاورید و آن را تحلیل کنید.

امتیازی: طراحی تابع پاداش جدید (Reward Shaping)

در نسخه پایه‌ی MountainCar، پاداش پیش‌فرض معمولاً sparse و کندآموز است. در این بخش باید دو تابع پاداش جدید برای این محیط طراحی و پیاده‌سازی کنید به‌طوری که یادگیری عامل را تسریع یا پایدارتر کند، بدون آنکه هدف اصلی مسئله یعنی رسیدن به پرچم عوض شود.

برای هر روش، باید تابع پاداش پیشنهادی خود را توضیح دهید، آن را در کد پیاده‌سازی کنید، مدل را حداکثر برای ۲۰۰۰ اپیزود آموزش دهید، و عملکرد روش خود را با پاداش پیش‌فرض محیط مقایسه کنید. معیار اصلی مقایسه، نرخ موفقیت در ۱۰۰ اپیزود اخیر است:

$$\text{success}[t] = \text{average}(\text{success}[t-99:t])$$

که در آن $\text{success}[t]$ برابر ۱ است اگر عامل در اپیزود t به پرچم برسد، و در غیر این صورت برابر ۰ است.

انتظار می‌رود روش‌های پیشنهادی شما نسبت به پاداش پیش‌فرض عملکرد بهتری داشته باشند. البته به دلیل نوپزی بودن آموزش در یادگیری تقویتی، آستانه عددی ثابتی برای نرخ موفقیت تعیین نمی‌شود و کیفیت روش‌ها بر اساس نمودارها، تحلیل و مقایسه‌ی آن‌ها با روش پایه بررسی خواهد شد.

همچنین باید یک نمونه Reward Hacking طراحی و پیاده‌سازی کنید. در این روش، عامل ظاهراً پاداش بیشتری می‌گیرد، اما رفتار یادگرفته‌شده با هدف اصلی مسئله یعنی رسیدن به پرچم سازگار نیست.

در نهایت، نمودار success rate را برای روش پایه، برای دو روش جدید خود، و نیز برای روش Reward Hacking رسم کنید و آن‌ها را به همراه تحلیل و مقایسه در گزارش خود بیاورید.

سوالات

۱. چرا برای استفاده از Q-learning کلاسیک باید فضای حالت را گسسته کنیم؟
۲. تاثیر افزایش و کاهش بیش از اندازه‌ی تعداد حالات گسسته در کیفیت سیاست‌ها، سرعت یادگیری و حافظه مصرفی را بررسی کنید.
۳. در روش Epsilon Greedy، عمل‌های غیرحریصانه چگونه انتخاب می‌شوند؟
۴. حداقل یک روش یا سیاست دیگر برای انتخاب عمل توسط عامل پیشنهاد دهید.

۵. در رابطه‌ی مربوط به به‌روزرسانی جدول Q، اگر مقادیر α و γ خیلی بزرگ یا خیلی کوچک انتخاب شوند، چه مشکلاتی ایجاد می‌شود؟
۶. توضیح دهید چرا استفاده از Epsilon ثابت (مثلاً ۰.۱) در بلندمدت مانع از رسیدن به امتیاز بهینه می‌شود
۷. چرا در حالت (۵×۵) عامل نمی‌تواند محیط را حل کند؟ (مفهوم Aliasing را بررسی کنید).
۸. چرا در حالت (۱۰۰×۱۰۰) سرعت یادگیری مدل به شدت کاهش می‌یابد؟ این موضوع چه ارتباطی با تعداد ملاقات (State Visitation) از هر حالت دارد؟
۹. با توجه به نمودار سه بعدی تابع ارزش، توضیح دهید چرا عامل در نقاطی که سرعت منفی دارد (در حال دور شدن از هدف به سمت تپه چپ) ارزش (Value) بالایی را پیش‌بینی می‌کند. این موضوع چگونه با فیزیک محیط (انرژی پتانسیل) در ارتباط است؟
۱۰. درمورد مفهوم Reward Hacking و انواع روش‌های آن تحقیق کنید و کاربرد آن در یادگیری تقویتی را توضیح دهید.

پاسخ این سوالات و مواردی که پیش‌تر عنوان شد را در گزارش خود بیاورید.

نکات پایانی

- در شرایط فعلی که دسترسی به اینترنت بین‌الملل نداریم، می‌توانید پکیج‌های پایتون موردنیاز خود را از [mirror](https://mirror-pypi.runflare.com/simple) دریافت کنید. برای مثال کافی‌ست دستور زیر را اجرا کنید:

```
pip install -i https://mirror-pypi.runflare.com/simple pygame
```


در صورت ابهام [این لینک](#) راهنما را مطالعه کنید.
- کد شما باید به نحوی زده شود که نتایج با پارامترهای مختلف قابلیت بازتولید داشته باشند.
- در این تمرین اولویت با نسخه‌ی Notebook است. اگر مشکلی در نصب داشتید یا به هر دلیلی اجرای آن ممکن نبود، می‌توانید از اسکریپت پایتون استفاده کنید.
- توضیحات مربوط به هر بخش از تمرین را خلاصه و مفید در گزارش ذکر کنید. حجم توضیحات هیچ تاثیری در نمره نخواهد داشت و تحلیل شماست که ارزشمند است.
- فایل‌های خود که شامل گزارش و کد می‌باشند را در قالب یک فایل فشرده با فرمت `AI_CA3_[stdNum].zip` در سامانه‌ی elearn بارگذاری کنید. به طور مثال `AI_CA3_810103127.zip`

- توجه کنید این تمرین باید به صورت تک‌نفره انجام شود و پاسخ‌های ارائه شده باید نتیجه‌ی فعالیت فرد نویسنده باشد. در صورت مشاهده تقلب به همه‌ی افراد مشارکت‌کننده، نمره‌ی ۱۰۰- داده شده و به استاد نیز گزارش می‌گردد. همچنین نوشته شدن کدها توسط هوش مصنوعی نیز بررسی میشود!
- سوالات خود را از طراحان تمرین از طریق ایمیل یا گروه درس بپرسید.

موفق باشید